

Лабораторная работа №

MLOps (Machine Learning Operations)

Теоретические сведения

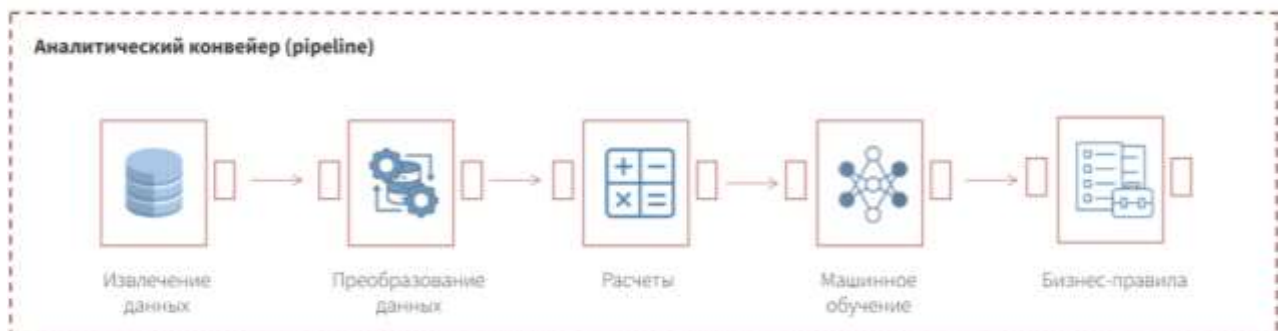
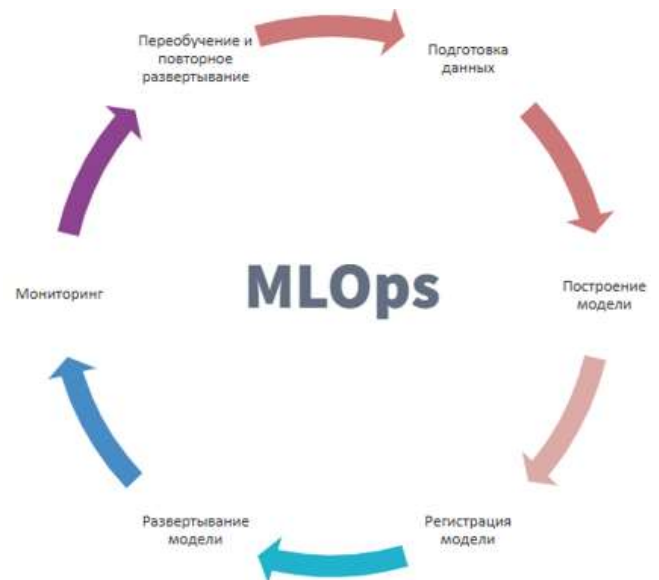
1. Операционализация аналитики данных

ModelOps - набор практик, направленных на унификацию процессов разработки и развертывания аналитических моделей и решений.

Компоненты MLOps:

- Трекинг экспериментов
- Управление данными
- Автоматизация обучения моделей
- Развертывание моделей
- Мониторинг моделей
- Непрерывная интеграция и развертывание CI/CD

ModelOps - это управление жизненным циклом любых аналитических процессов, а не только моделей машинного обучения.



Ключевые требования к бизнес-процессу:

- В рамках одной команды/проекта важно обеспечить воспроизводимость вычислений
 - Коллеги должны иметь возможность повторить эксперимент без особых усилий
- Особенности аналитических процессов:

- Требование воспроизводимости
 - Несколько версий датасетов
 - Несколько вариантов решения задачи
 - Трудность выбора лучшего варианта сразу
 - Необходимость сравнить два эксперимента между собой
- «Подводные камни» воспроизводимости:

- Различия в среде исполнения (разработка, тестирование, продакшн)
- Неуправляемая случайность в данных или алгоритмах (Random Seed)
- Проблемы с зависимостями со ссылками-пакетами, библиотеками языков программирования, зависимости в зависимостях

2. Эксперименты и модели: уровни зрелости

В эпоху данных модели критичны для бизнеса и аналитических процессов- Зрелость управления экспериментами и моделями определяет эффективность использования аналитических процессов. Процесс развития управления экспериментами и моделями можно условно разделить на несколько уровней (см. рисунок 1).



Рис. 1. Уровни Зрелости MLOps

Уровень 1: нет автоматизации. На старте, работа над моделями часто ведется обособленно, данные разрознены, а версии отслеживаются вручную. Это приводит к низкой эффективности, высокому риску ошибок и сложностям в масштабировании аналитики. Решением становится внедрение базовых инструментов: системы контроля версий для кода, централизованного репозитория для данных и, конечно, стандартизация рабочих процессов.

Уровень 2: частичная автоматизация. Следующий этап — автоматизация рутинных задач. Создаются централизованные репозитории, внедряется версионирование и конвейеры CVCD для автоматической сборки и тестирования моделей. Результат — повышение эффективности и снижение рисков. Однако, остаются проблемы: автоматизация все еще не полная, мониторинг поверхностный, а масштабируемость ограничена. Для дальнейшего прогресса необходима более глубокая автоматизация, внедрение инструментов для отслеживания экспериментов, и улучшение мониторинга моделей в реальном времени.

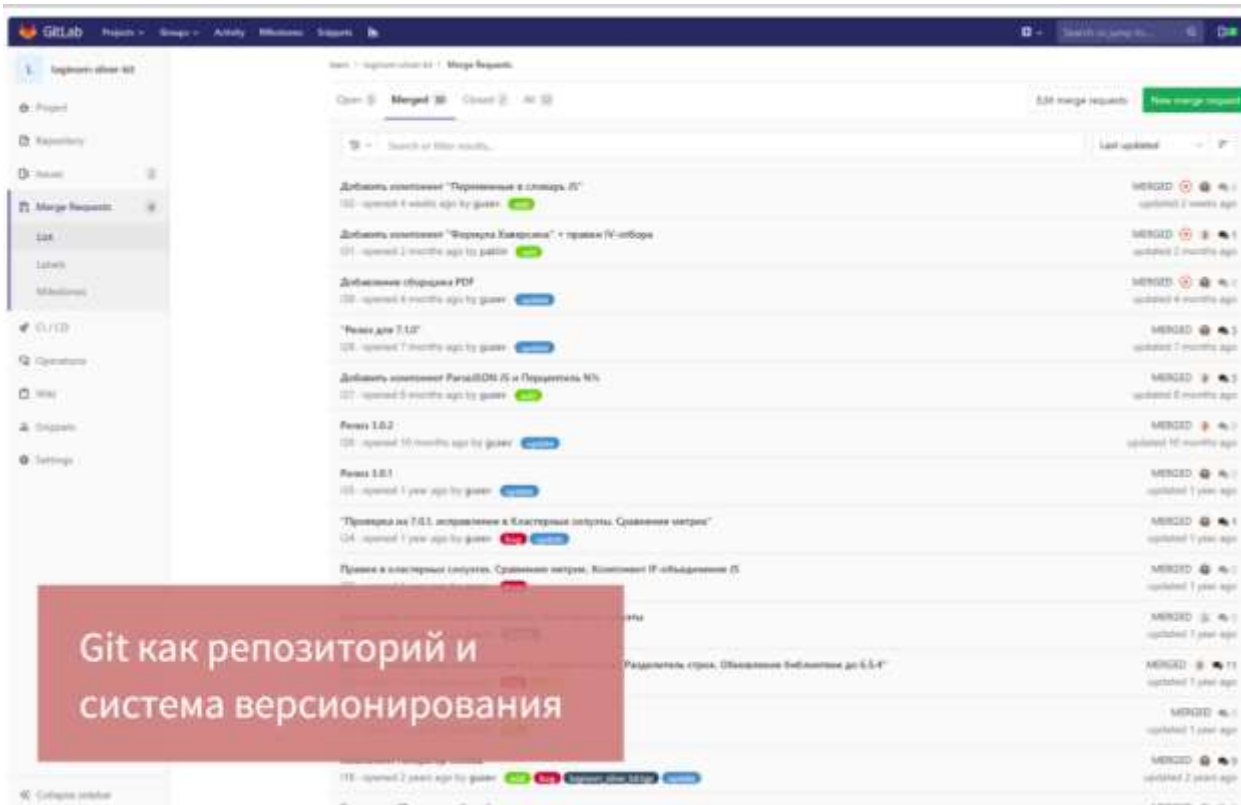
Уровень 3: значительная автоматизация. На вершине зрелости находятся организации, внедрившие фреймворки ModelOps. Они автоматизируют весь жизненный цикл моделей, от подготовки данных до развертывания и мониторинга, используя контейнеризацию и оркестрацию. Это обеспечивает максимальную эффективность, качество, масштабируемость и полную прозрачность аналитических процессов. Минусы — высокая стоимость и сложность внедрения, но преимущества значительно перевешивают.

3. Git в мире MLOps: возможности и ограничения

Git — это распределенная система контроля версий, самая популярная в мире, которая позволяет отслеживать изменения в файлах (обычно в коде), совместно работать над проектами и восстанавливать предыдущие состояния проекта.

Git, изначально разработанный для управления версиями кода, стал неотъемлемой частью MLOps при использовании языков программирования. Он обеспечивает

совместную разработку, отслеживание изменений и воспроизводимость экспериментов, но подходит ли он для всего в MLOps, да еще и в контексте low-code?



Сначала рассмотрим преимущества использования Git для процессов MLOps.

- *Контроль версий файлов.* Основная функция Git и его главное преимущество. Это критически важно для воспроизводимости экспериментов и отладки.
- *Воспроизводимость.* Git позволяет вернуться к любому предыдущему состоянию проекта, восстановить конкретную версию кода, конфигурации или даже данных.
- *Широкая распространенность и зрелая экосистема.* Git - де-факто стандарт в разработке программного обеспечения, поэтому существует огромная база знаний, множество инструментов и ресурсов для его использования.
- *Совместная работа.* Возможность нескольким разработчикам работать над одним проектом одновременно, разрешая конфликты и обеспечивая согласованность изменений.
- *Интеграция с другими инструментами.* Git легко интегрируется с различными инструментами CVCD, отслеживания экспериментов и фреймворками для MLOps

Выделим главные недостатки применения Git для MLOps:

- *Ограничения на размер файлов.* Git изначально не предназначен для хранения больших файлов, таких как большие датасеты, обученные модели или артефакты экспериментов.
- *Не подходит для хранения версионированных данных.* Git не оптимизирован для хранения и версионирования больших структурированных данных, которые часто используются в машинном обучении.
- *Не оптимален для хранения метаданных моделей.* Хотя Git может хранить метаданные моделей (например, параметры обучения, метрики производительности), это не самый эффективный способ.
- *Высокий порог входа для аналитиков и low-code специалистов.* Работа с Git требует понимания концепций ветвления, слияния, коммитов и т.д., что может быть сложным для

аналитиков и low-code разработчиков, привыкших к визуальным интерфейсам. Это может замедлить процесс разработки и потребовать дополнительных усилий на обучение.

Git - мощный инструмент для контроля версий кода и совместной работы в MLOps, но он не является универсальным решением для всего. Для low-code окружений, где требуется более простой и визуальный подход к управлению версиями, следует рассмотреть альтернативные инструменты.

4. Фреймворки для MLOps

Итак, фреймворки для MLOps представляют собой набор инструментов и платформ, которые помогают компаниям эффективно управлять жизненным циклом аналитического процесса, начиная от разработки и заканчивая развертыванием и мониторингом их исполнению. Самое популярное применение - для машинного обучения.

Выбор подходящего фреймворка для MLOps — ключевое решение для любой компании, стремящейся эффективно управлять жизненным циклом моделей машинного обучения. Существует множество решений, и оптимальный выбор зависит от специфических потребностей и ограничений. Ниже мы рассмотрим ключевые критерии по выбору Фреймворка, особенно акцентируя внимание на потребностях аналитиков и low-code разработчиков.

1. *Наличие облачной версии.* Возможность начать работу с фреймворком без значительных финансовых вложений — важный фактор, особенно для небольших команд или стартапов. Наличие бесплатной облачной версии, пусть и с ограничениями по ресурсам или функциональности, позволяет протестировать фреймворк и оценить его пригодность.

2. *Работа в локальной сети.* Многие организации, особенно в секторах с высокими требованиями к безопасности и конфиденциальности данных, нуждаются в возможности развертывания MLOps-инфраструктуры в локальной сети (on-premise). Фреймворк должен поддерживать развертывание как в облаке, так и в локальной инфраструктуре.

3. *Поддержка сторонних файловых хранилищ.* Эффективная работа с данными предполагает возможность интеграции с различными типами файловых хранилищ, будь то облачные решения или локальные файловые системы. Фреймворк должен обеспечивать прозрачный доступ к данным, независимо от их расположения.

4. *Подходит для систем класса low-code.* Фреймворк должен быть адаптирован для работы с low-code платформами, позволяя аналитикам и бизнес-пользователям, не имеющим опыта программирования, участвовать в процессе разработки и развертывания моделей.

5. *Низкий порог входа для аналитика.* Интерфейс Фреймворка должен быть интуитивно понятным и простым в использовании, даже для аналитиков, не имеющих опыта работы с визуальными инструментами, подробная документация и примеры использования — важные факторы снижения порога входа.

6. *Богатый REST API.* Наличие богатого и хорошо документированного REST API позволяет интегрировать фреймворк с другими системами и инструментами, автоматизировать процессы и создавать собственные расширения. Это необходимо для построения гибкой и масштабируемой MLOps-инфраструктуры.

Сегодня доступно несколько популярных фреймворков для MLOps: MLFlow, TensorBoard, VertaAI, Weight&Biases. В таблице ниже представлено сравнение этих платформ по ключевым функциональным возможностям; «+» означает, что платформа предоставляет данную функциональность, «-» — что функциональность отсутствует или реализована в ограниченном виде.

Опция	MLFlow	ClearML	Tensor Board	W&B	Verta AI
Локальное хранение	+	+	+	+	+
Менеджер экспериментов	+	+	+	+	+
Менеджер датасетов	-	+	-	-	
Единая платформа	-	+	-	-	

Рис. 2. Сравнительная таблица фреймворков

Анализируя представленную таблицу, можно сделать вывод, что ClearML выделяется как наиболее комплексное решение, предоставляющее интегрированный менеджер датасетов и систему управления задачами. Ближайшим конкурентом по функциональности является Weights & Biases (W&B), обладающий мощными инструментами для отслеживания экспериментов и визуализации результатов. Однако, не является единой платформой в том же смысле, что и ClearML, поскольку требует интеграции с другими инструментами для управления датасетами и оркестрации задач.

Кроме того, ClearML, в отличие от Weights & Biases, является решением класса open source с возможностью развертывания внутри корпоративной сети, что особенно актуально для нашей страны, где есть риски недоступности зарубежного проприетарного ПО. Также, акцент ClearML на автоматизации и управлении задачами, а не на интерактивной визуализации, делает её более пригодной для интеграции с low-code платформами, позволяя расширить возможности этих систем и обеспечить гибкость в управлении аналитическими процессами.

5. Обзор ClearML

ClearML — это комплексная платформа с открытым исходным кодом, предназначенная для управления всем жизненным циклом аналитического процесса и машинного от экспериментов до развертывания моделей.

ClearML предлагает модульную архитектуру, охватывающую различные аспекты жизненного цикла машинного обучения. Ниже мы сосредоточимся на ключевых модулях, посвященных управлению моделями, датасетами и экспериментами, которые составляют основу MLOps.



Рис. 3 Модули ClearML



Рис. 4 Управление моделями и экспериментами

Модуль **DataOps** предназначен для управления версиями датасетов, отслеживания их происхождения и обеспечения консистентности данных, используемых в моделях. Этот модуль позволяет создавать снимки датасетов, отслеживать изменения и связи между различными версиями данных. Это критически важно для воспроизводимости экспериментов. Важно отметить, что ClearML фокусируется на управлении версиями и lineage датасетов, а не на их хранении. Реальные данные хранятся в сторонних хранилищах, а ClearML хранит метаданные и ссылки на эти данные.

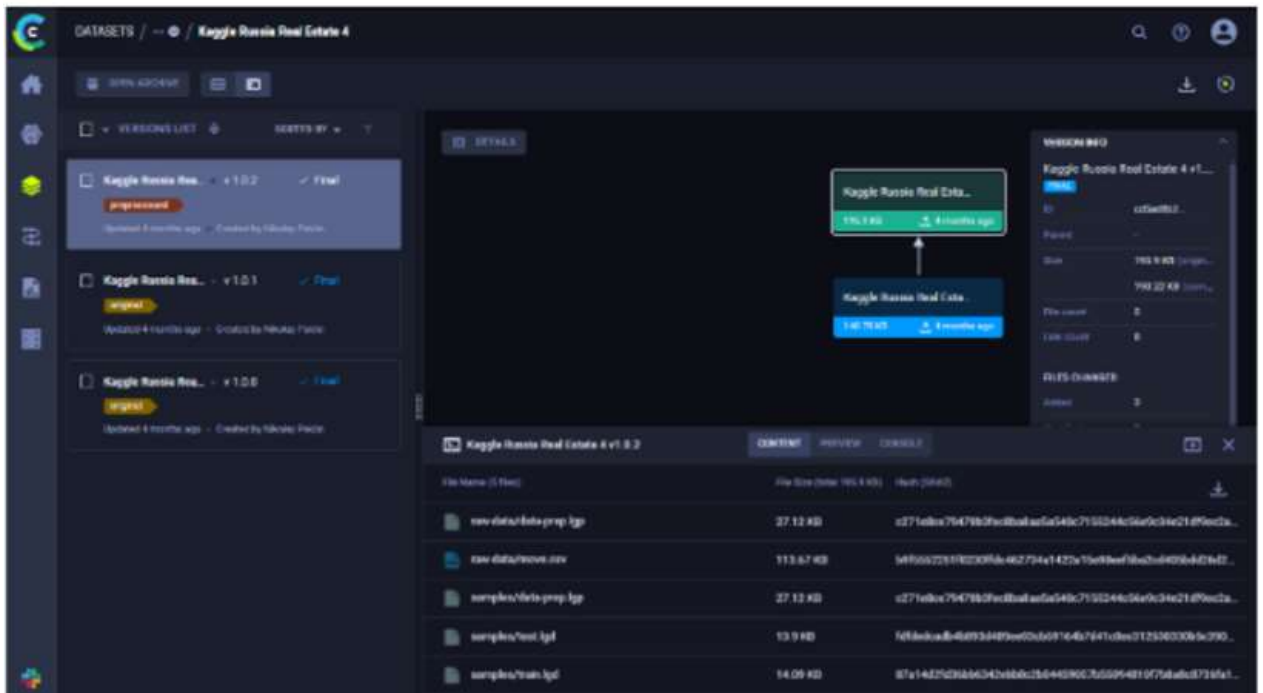


Рис. 5. Модуль DataOps в ClearML

По умолчанию, в ClearML для хранения метаданных используется PostgreSQL.

Модуль **Experiment** является сердцем ClearML для отслеживания, сравнения и воспроизведения экспериментов машинного обучения. Он позволяет записывать параметры, метрики, артефакты (модели, графики, файлы) и исходный код, обеспечивая полную прозрачность и воспроизводимость экспериментов. С помощью этого модуля можно легко сравнивать результаты различных экспериментов, находить оптимальные параметры обучения и анализировать влияние различных факторов на производительность моделей.

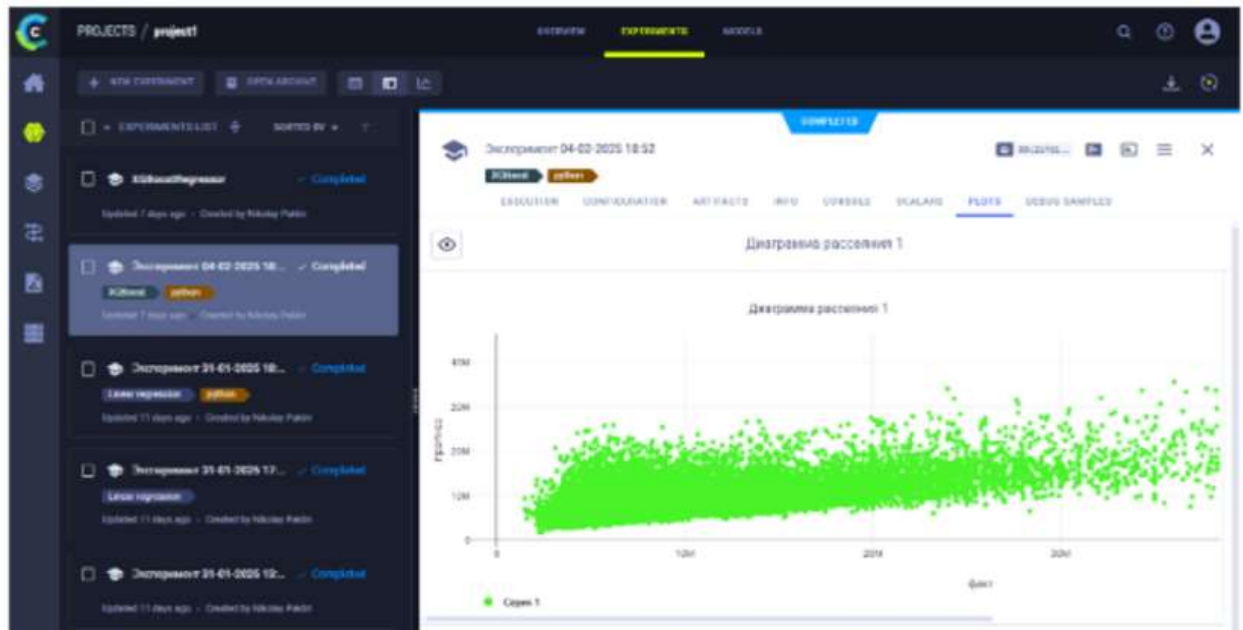


Рис. 6. Модуль Experiment в ClearML

Модуль **Train** отвечает за исполнение экспериментов. Это не просто запуск скрипта; модуль Train автоматически логирует все важные аспекты эксперимента (параметры, метрики, код, артефакты) в ClearML Server.

Модуль **ModelStore** обеспечивает централизованное управление моделями машинного обучения. Он позволяет регистрировать, версионировать, развертывать и отслеживать производительность моделей в продакшене. С помощью этого модуля можно легко переключаться между различными версиями моделей, проводить A/B тестирование и мониторить их состояние. Важной особенностью модуля является возможность автоматического развертывания моделей в различные окружения.

Модуль **Reports** позволяет создавать интерактивные отчеты и дашборды на основе данных, собранных в ClearML. Он дает возможность визуализировать результаты экспериментов, анализировать производительность моделей и делиться этим с другими команды. Этот модуль облегчает коммуникацию и принятие решений на основе данных, а также позволяет создавать отчеты о соответствии нормативным требованиям. Reports могут включать графики, таблицы, текстовые описания и другие элементы визуализации, а также ссылки на конкретные эксперименты, датасеты и модели.

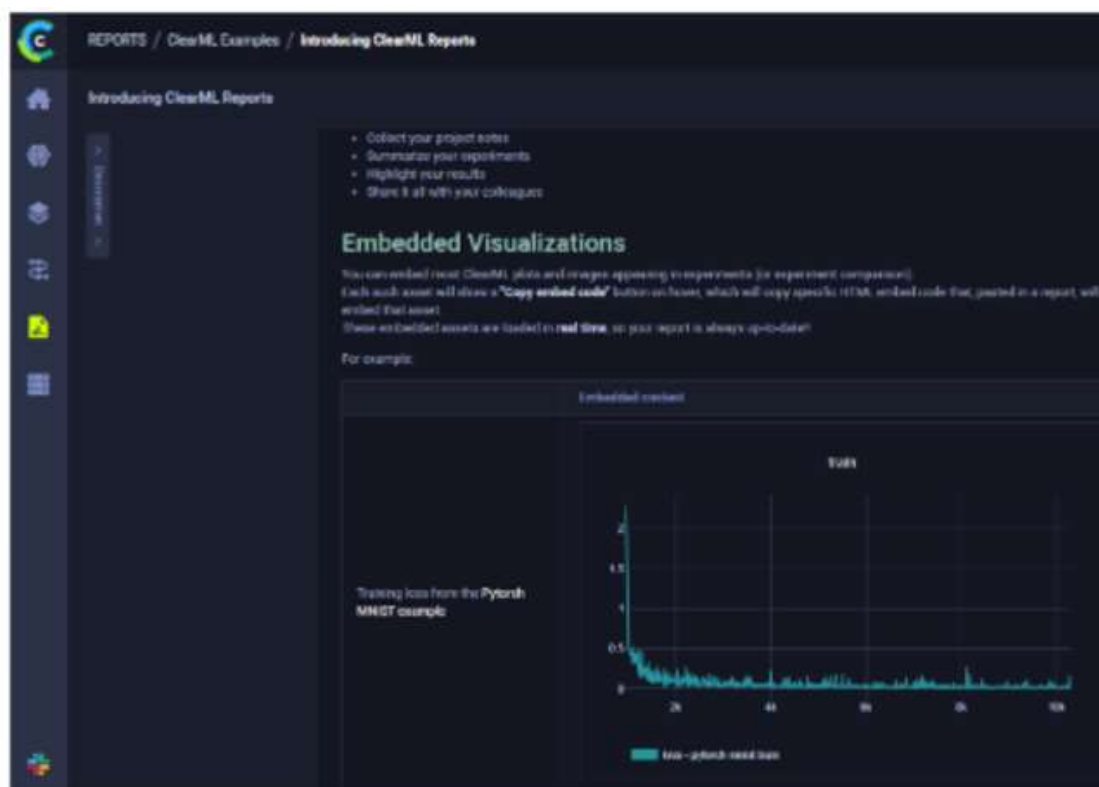


Рис. 7. Пример отчета в ClearML

6. Основные сущности ClearML

ClearML, как платформа для управления жизненным циклом аналитического процесса, оперирует с рядом основных сущностей, которые представляют собой ключевые элементы MLOps. Понимание этих сущностей необходимо для эффективного использования ClearML и организации ваших ML-проектов.

Проект (Project)

Проект — это контейнер верхнего уровня для организации всех связанных задач, экспериментов, моделей и данных. Он представляет собой логическую группировку,

позволяющую структурировать работу над и обеспечивать совместный доступ к ресурсам для команды. Проект позволяет контролировать доступ, устанавливать общие стандарты и обеспечивает согласованность в рамках всей команды аналитиков. По сути, это ваша папка, где лежат все компоненты MLOps для конкретной задачи.

Датасет (Dataset)

Датасет в ClearML представляет собой версионированный набор данных, используемый для обучения, валидации и тестирования моделей. ClearML не хранит сами данные, а управляет метаданными о датасете, включая его происхождение (lineage), версии и связи с экспериментами и моделями. Это обеспечивает воспроизводимость экспериментов и упрощает процесс отслеживания изменений в данных. Каждый раз, когда выделаете изменения с данными, создается новая версия датасета, что позволяет вернуться к любому состоянию.

Эксперимент (Experiment)

Эксперимент — это основной элемент CleanML, представляющий собой запуск аналитического процесса (в Loginom-пакета `lgp`). ClearML автоматически отслеживает все важные аспекты эксперимента: параметры, метрики, артефакты (модели, графики, лог) и исходный код. Это обеспечивает полную воспроизводимость экспериментов и упрощает процесс сравнения различных подходов. Каждый эксперимент привязан к конкретному проекту и может быть связан с датасетами и моделями.

Модель (Model)

Модель — это обученный артефакт, полученный в результате эксперимента. ClearML позволяет регистрировать, версионировать и развертывать модели, а также отслеживать их производительность в продакшене. Модели связываются с экспериментами, которые их создали, и могут быть развернуты для предсказаний.

В Loginom обученная модель содержится в самом пакете со сценарием обработки. Если в процессе вычислений использовался Python, то модель представляет собой файл с расширением `pkl` или `joblib`

Гиперпараметр (Hyperparameter)

Гиперпараметры — это параметры, которые используются для управления аналитическим процессом обработки. ClearML отслеживает значения гиперпараметров для каждого эксперимента- Это может быть любая настройка или параметр, который аналитик считает важным для отслеживания и сравнения экспериментов, даже если это не традиционный гиперпараметр модели, а, например, параметр предобработки данных или определенная настройка компонента Loginom.

В классическом понимании, гиперпараметры модели - это параметры, которые задаются до начала обучения модели и не изменяются в процессе обучения (например, скорость обучения, количество слоев в нейронной сети, глубина дерева решений).

Метрика (Metric)



Метрики — это численные показатели, для оценки производительности модели (например, точность, полнота, F1-мера, AUC). ClearML отображает метрики на отдельной вкладке и позволяет визуализировать их в виде графиков, упрощая процесс анализа и сравнения экспериментов.

Артефакт (Artifact)

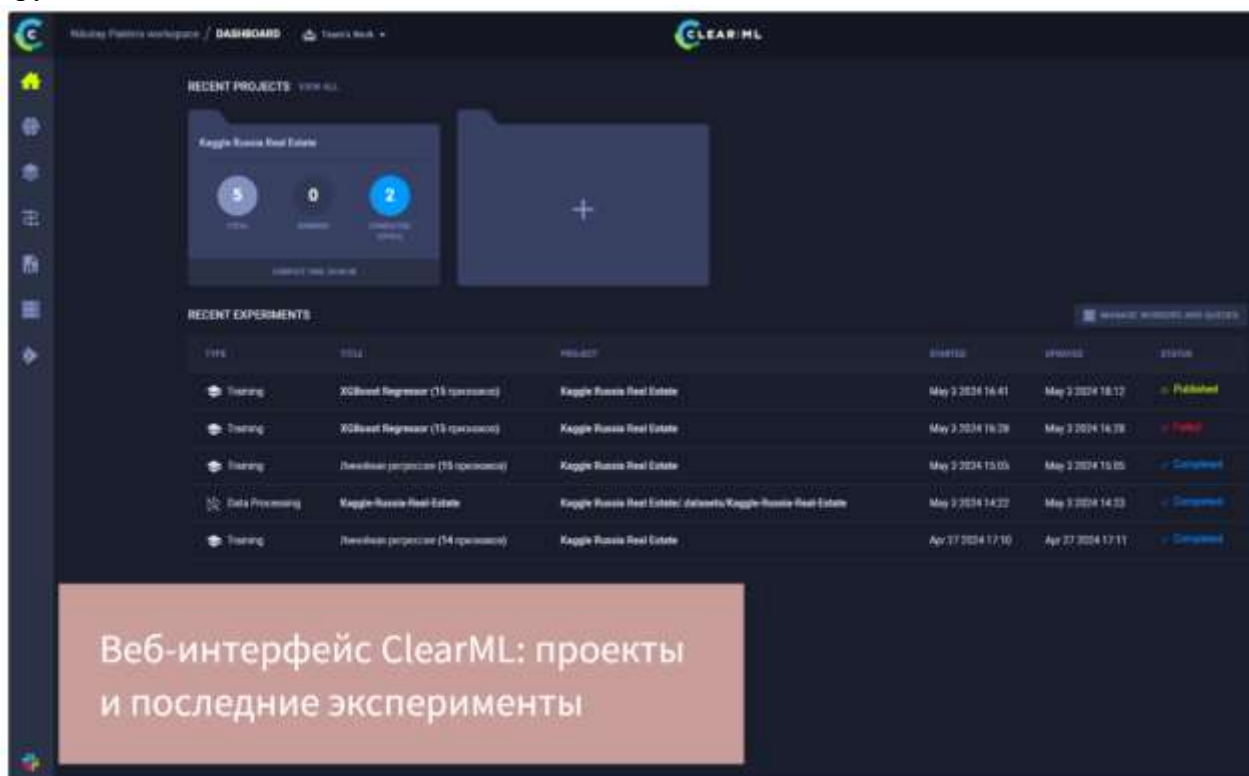
Артефакты — это файлы, сгенерированные в результате эксперимента (например, обученные модели, графики, лог, таблицы с результатами). ClearML позволяет хранить и версионировать артефакты, обеспечивая доступ к ним в любой момент времени.

Отчет (Report)

Отчеты позволяют собрать и систематизировать информацию об экспериментах, моделях и данных в удобном для просмотра и анализа формате. Отчеты могут включать графики, таблиц, текстовые описания и другие элементы визуализации, и позволяют легко делиться результатами работы с другими членами команды.

Опубликованный объект (Published Object)

Опубликованные объекты — это артефакты, эксперименты или модели, которые были специально помечены как готовые для использования. Этот статус означает, что объект был проверен, протестирован и одобрен для использования в продакшене или для других целей.



Практическое задание

1. Подготовка к работе

Для выполнения работы, потребуется версия Loginom не ниже 7.2.x;

Если у вас установлен Loginom, проверьте версию. Установить последнюю версию Loginom можно по ссылке: <https://loginom.ru/download>

Установка Python

- Скачайте последний стабильный дистрибутив для вашей ОС с сайта и запустите инсталлятор (<https://www.python.org/downloads/>)
- Запустите Loginom и откройте Администрирование - Параметры
- Настройте параметры компонента Python. Пример заполненных параметров приведен на рисунке

Параметры компонента: Python	
Выполнение запрещено	false
Путь библиотеки	C:\Program Files\Python39\python39.dll
Путь интерпретатора	C:\Program Files\Python39\python39.exe

Параметры компонента Python

Установка библиотек Python

Для запуска сценария воркшопа понадобится ряд библиотек Python:

- **clearml** – не ниже 1.16.5 (<https://pypi.org/project/clearml/>)
- **lxml** – актуальная версия (<https://pypi.org/project/lxml/>)
- **pandas** версии не ниже 1.5 (<https://pandas.pydata.org/>)
- **scikit-learn** - актуальная версия (<https://scikit-learn.org/>)
- **category-encoders** - актуальная версия (https://contrib.scikit-learn.org/category_encoders/)
- **xgbost** - актуальная версия (<https://xgboost.readthedocs.io/en/stable/index.html>)

По ссылкам можно найти документацию к библиотекам.

Самый простой способ установить библиотеки – запустить командную строку и набрать команду:

pip install <название библиотеки>

Список всех установленных библиотек с версиями можно получить командой: `pip list`. Для обновления библиотеки до последней версии используйте команду:

pip install <название библиотеки> --upgrade

Установка библиотек Loginom

Кроме библиотек Python, нам понадобится библиотека готовых компонентов *Loginom Python kits* - компоненты для встраивания Python -логики в сценарий Loginom без кодинга. Интеграция с ClearML также сделана через Python.

Библиотека Loginom Python kits должна быть не ниже 3.2.2. Скачать ее можно по ссылке. Там же приведены инструкции по установке.
<https://marketplace.loginom.ru/libraries/loginom-python-kits/>

Установка сводится к тому, чтобы положить папки с пакетами библиотек в общую папку `libs`. Сценарии, которые вы открываете, должны находиться в какой-то папке на одном уровне с папкой `libs`, например `workshop-mlops`. В противном случае сценарий откроется с ошибками - пакеты библиотек не будут найдены по нужным путям.

ClearML

ClearML — это платформа для управления машинным обучением, которая позволяет отслеживать эксперименты, модели и метрики. Она доступна как в виде облачного сервиса, так и в виде сетевого на собственном сервере. Рассмотрим оба варианта.

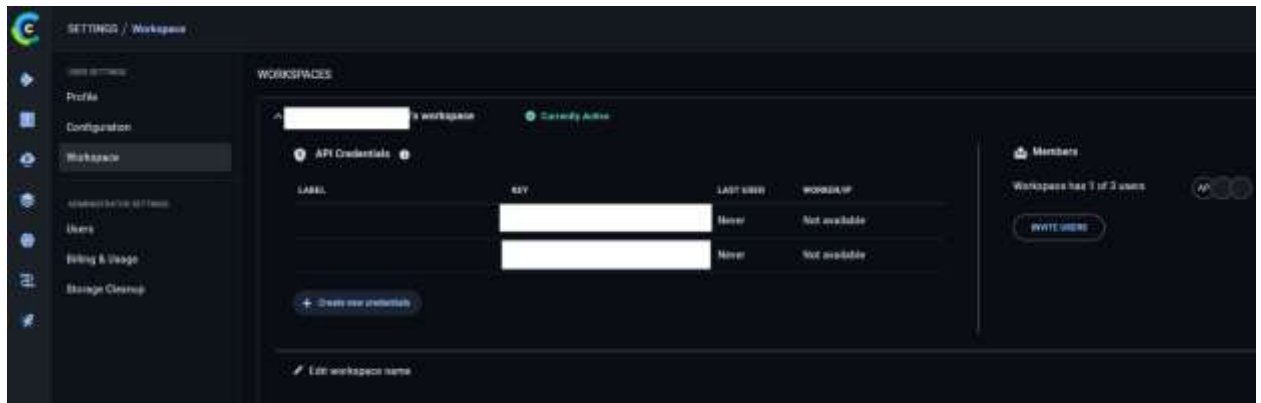
Регистрация в облачном сервисе ClearML

Шаг 1: Создание аккаунта

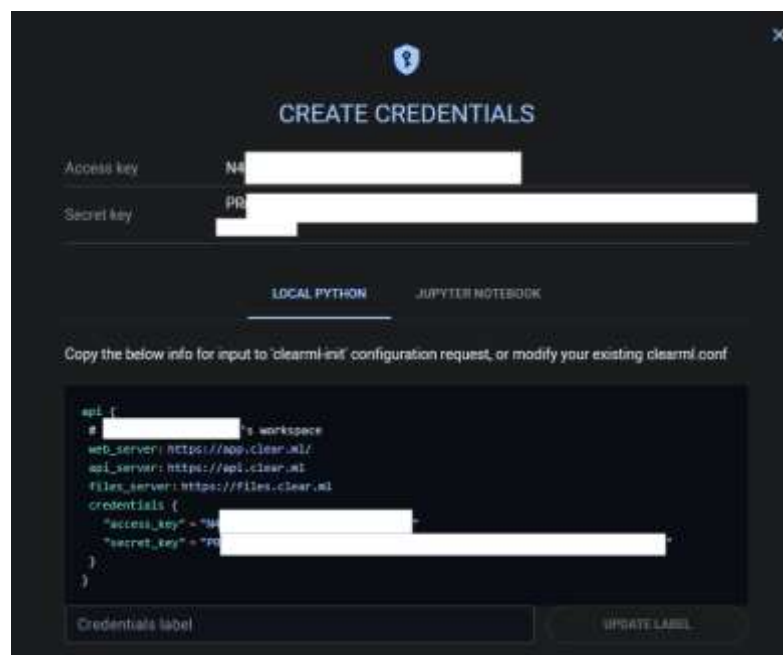
1. Перейдите на сайт ClearML (<https://app.clear.ml/>)
2. Нажмите кнопку Free Sign Up (Зарегистрироваться).

Шаг 2: Получение API ключей

1. После регистрации и входа в систему на сайте перейдите в *Settings* в верхнем правом углу страницы (иконка профиля).
2. В открывшемся меню выберите вкладку *Workspace*.
3. Нажмите на + *Create new credentials*



4. Скопируйте *Access Key* и *Secret Key*. Также рекомендуем сохранить адреса: *web_server*, *api_server*, *files_server* для того чтобы указать их в переменных компонента или в конфигурационном файле `clearml.params`.



Теперь вы готовы к работе с ClearML, используя компоненты ClearML kit. У вас 100 Гб свободного пространства, 1 млн. API-вызовов в месяц и максимум 3 пользователя в команде проекта.

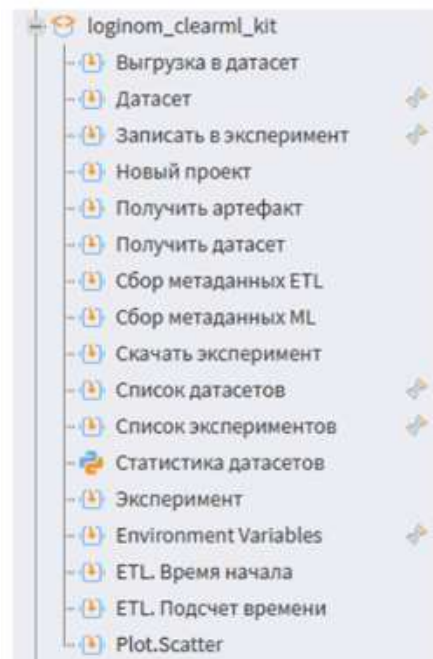
2. Практическое задание

Для облегчения работы с ClearML в Loginom разработан набор компонентов, который входит в состав Loginom Python Kits; - библиотеки компонентов на основе языка программирования Python.

Компоненты позволяют прямо из сценария Loginom без кодирования отправлять результаты экспериментов в фреймворк ClearML.

В каждом сценарии Loginom, где вы планируете отправлять данные в ClearML, должна быть подключена библиотека компонентов ClearML kit из набора библиотек Loginom Python Kits- для подключения проделайте следующие шаги:

1. В Loginom Создайте новый пакет или откройте существующий.
2. В дереве объектов проекта перейдите в *Ссылки*
3. Подключите пакет `loginom_clearml_kit.lgp`.



+ Добавить ⚙ Настроить 🗑 Удалить		
Имя	Путь	Версия
loginom_clearml_kit	../loginom-python-kits-master/loginom-python-kits-master/python_kits/loginom_clearml_kit.lgp	^3.2.2

Быстрый старт

Для того чтобы максимально упростить использование компонентов, мы разработали шаблоны сценария для двух самых популярных задач обработки данных: *ETL-процесс* и *построение ML-модели*.

Шаблон представляет собой готовый сценарий из компонентов библиотеки ClearML kit, который нужно наполнить реализующих тот или иной аналитический процесс и сформировать метаданные, которые будут отправляться в эксперимент фреймворка ClearML. Все остальные операции по подключению к фреймворку уже сформированы готовыми компонентами в шаблоне и связями между собой.

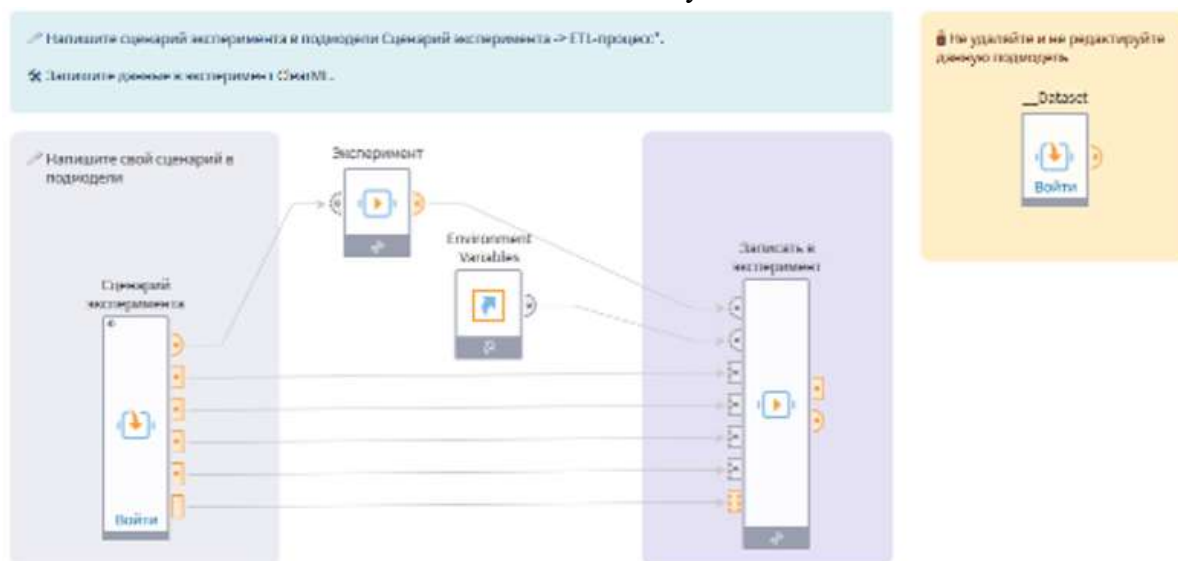


Рис. 8. Пример шаблона сценария

Для быстрого старта рекомендуем придерживаться следующей последовательности действий.



Рис. 9. Порядок работы.

1. Найдите в папке `/data` директории `python_kits` файл `clearml.params`. Он имеет следующий вид

```

api {
  web_server:null
  api_server:null
  files_server:null
  credentials {
    "access_key"="null"
    "secret_key"="null"
  }
}

```

В секцию `credentials` вставьте ваши секретные ключи доступа и сохраните файл.

Если файл `clearml.params` не будет найден, то реквизиты доступа к CleanML придется задавать каждый раз в компоненте `Environment Variables`.

2. Создайте новый проект как черновик и подключите пакет `loginom_clearml_kit.lgp` из папки `python_kits`.

3. Добавьте в сценарий компонент **Новый проект** и откройте настройки входного порта переменных.

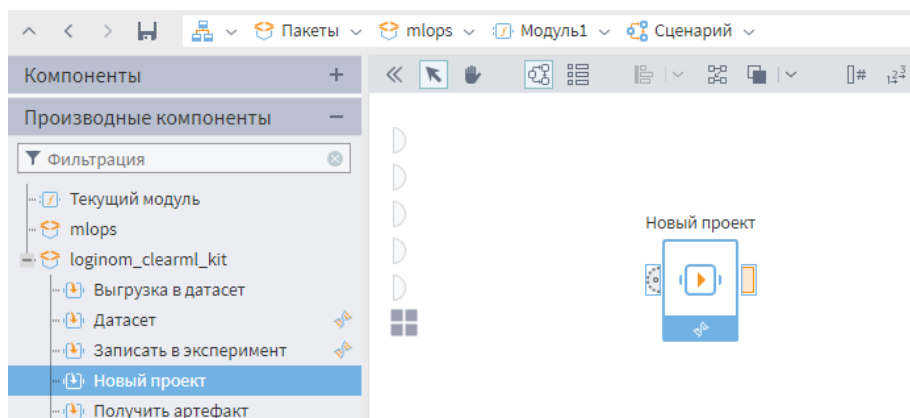


Рис. 10. Компонент Новый проект

Настройка переменных

Метка	Имя	Назначение	Значение
ab Расположение	target_folder	Не задано	<null>
ab Проект	project_name	Не задано	Новый проект
ab Тип проекта	project_type	Не задано	ml
ab Имя папки с данными	data_folder_name	Не задано	raw-data

Рис. 11. Настройка входных переменных

4. Отредактируйте переменные **Проект** и **Тип проекта**. Тип проекта может принимать два значения- **etl** и **ml**. В результате выполнения компонента будет создана папка с именем проекта и заполнена набором фатов, список которых зависит от типа проекта.

Также отредактируйте переменную **Расположение**. В этой переменной будет храниться путь до рабочего каталога пользователя. Это необходимо для создания папки с проектом и дальнейшей работой с ней.

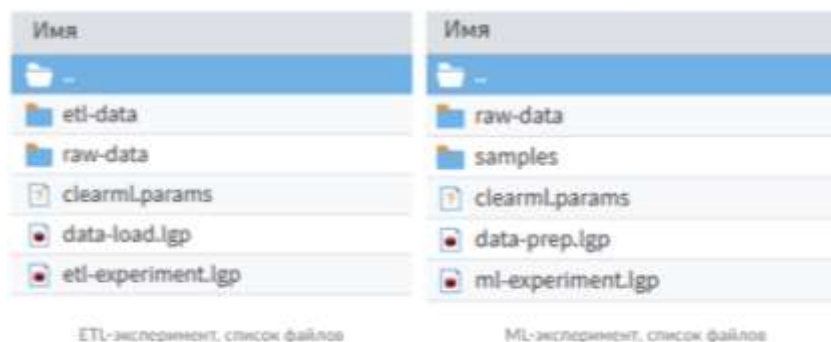
Настройка переменных

Метка	Имя	Назначение	Значение
ab Расположение	target_folder	Не задано	C:\
ab Проект	project_name	Не задано	project1
ab Тип проекта	project_type	Не задано	etl
ab Имя папки с данными	data_folder_name	Не задано	raw-data

5. Нажмите **Выполнить** узел и перейдите в файловое хранилище, найдите папку с проектом. Эталонные файлы хранятся в каталоге **/data** библиотеки компонентов Python Kits.

#	ab Параметр	ab Значение
1	Успех	Создана папка: C:\Users\ \ project1
2	Успех	Создана папка: C:\Users\ \ project1\raw-data
3	Успех	Создана папка: C:\Users\ \ project1\etl-data
4	Успех	Скопирован файл: data-load.lgp
5	Успех	Скопирован файл: etl-experiment.lgp
6	Успех	Скопирован файл: clearml.params

Результат выполнения узла



На рисунке вы видите набор файлов, который формируется в зависимости от типа проекта.

В результате выполнения узла в рабочем каталоге пользователя создается папка с именем проекта. Обратите внимание, что дальнейшая работа ведется с файлами из созданной папки с проектом.

Имя	Дата изменения	Тип	Размер
etl-data	28.04.2025 17:49	Папка с файлами	
raw-data	28.04.2025 17:49	Папка с файлами	
clearml.params	08.04.2025 13:17	Файл "PARAMS"	1 КБ
data-load.lgp	05.04.2025 15:49	Loginom Package...	22 КБ
etl-experiment.lgp	05.04.2025 15:49	Loginom Package...	63 КБ

Созданная папка с файлами проекта

ETL-эксперимент

ETL-эксперимент представляет собой какую-либо последовательность обработки по преобразованию датасета (набора датасетов).

Создав новый проект с типом etl , поместите в подпапку которые будут основой для вашего. Если их нет – используются внешние подключения к источникам данных; данный шаг не является обязательным.

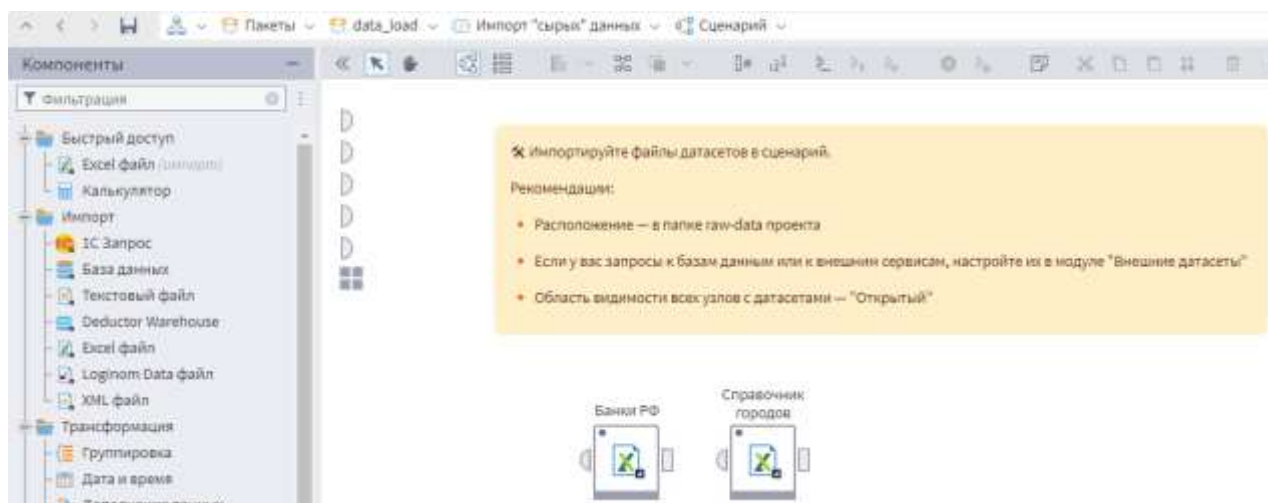
1. Подготовка датасетов. Откройте пакет *data-load.lgp*. Настройки по умолчанию можно оставить как есть.

Выводные переменные

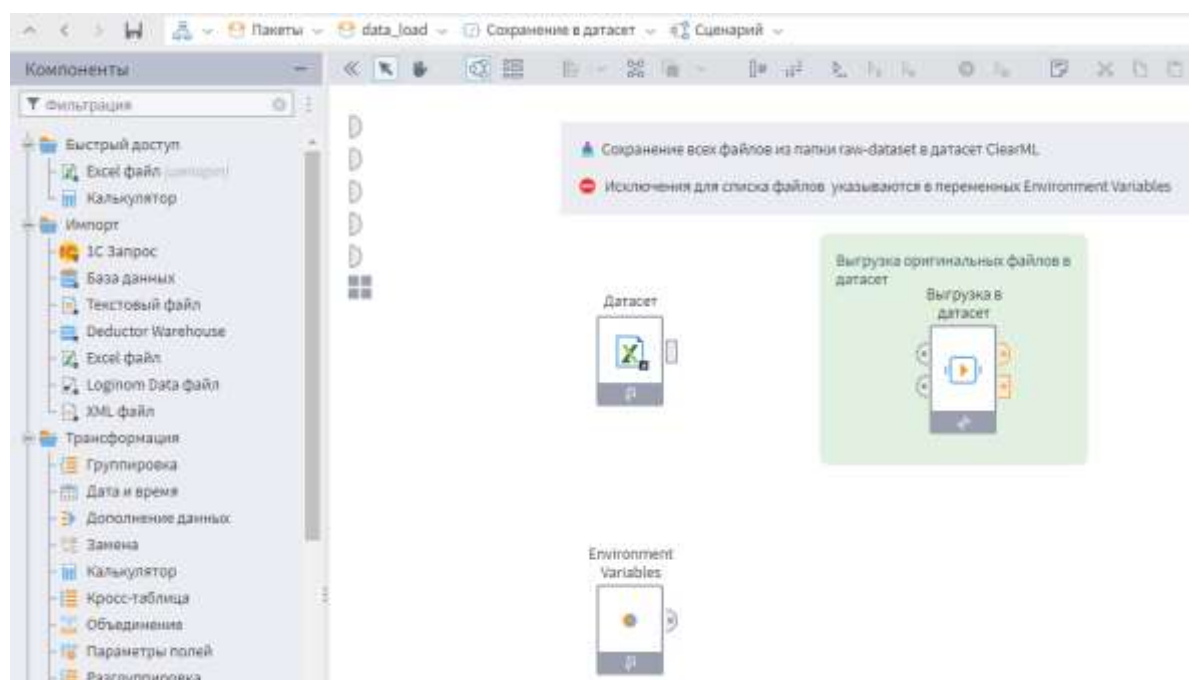
№	Имя	Метка	Значение
1	ab project_name	Имя проекта	project1
2	ab dataset_name	Датасет	project1_dataset
3	ab dataset_id	Id датасета	<null>
4	ab parent_dataset	Родительский датасет	<null>
5	ab version	Версия	<null>
6	ab tags	Тэги	<null>
7	ab Comment	Описание датасета	<null>

2. Перейдите в модуль **Импорт «сырых» данных.**

Импортируйте все датасеты из папки **mlors** и установите у них модификатор доступа **Открытый**.



Сохраните сценарий и перейдите в модуль **Сохранение в датасет.**



В параметрах узла установите ссылку на библиотеку **Loginom Python Kits**.

Выполнение узла



Отредактируйте верхний порт входных переменных узла **Выгрузка в датасет**. Пример заполнения представлен на рисунке ниже

Настройка переменных

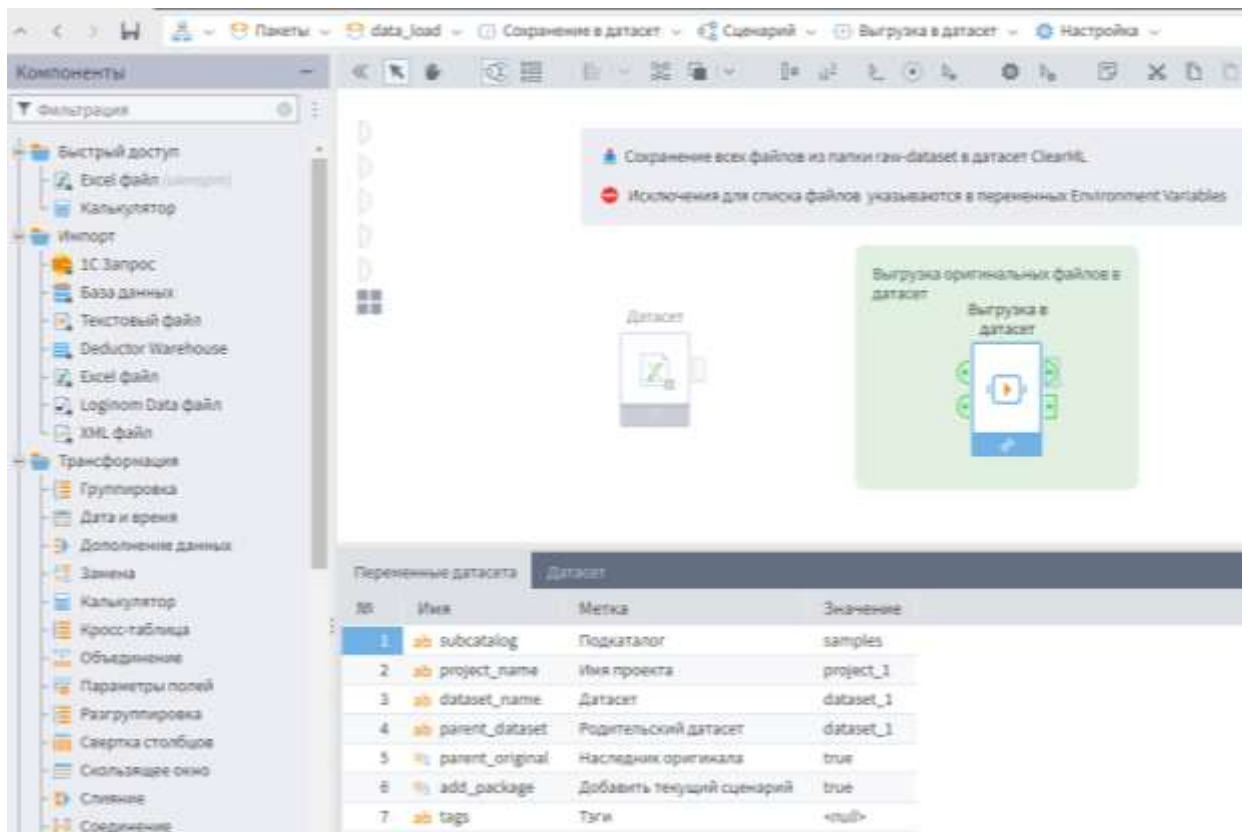
Метка	Имя	Назначение	Значение
Подкаталог	subcatalog	Не задано	raw-data
Имя проекта	project_name	Не задано	project_1
Датасет	dataset_name	Не задано	dataset_1
Версия	version	Не задано	v1
Родительский датасет	parent_dataset	Не задано	<null>
Наследник оригинала	parent_original	Не задано	true
Тэги	tags	Не задано	<null>
Добавить текущий сценар...	add_package	Не задано	true
Описание	Comment	Не задано	<null>

Отредактируйте нижний порт входных переменных узла **Выгрузка в датасет**. Пример заполнения представлен на рисунке ниже.

Настройка переменных

Метка	Имя	Назначение	Значение
Ключ доступа	CLEARML_API_ACCESS_KEY	Не задано	L
Секретный ключ	CLEARML_API_SECRET_KEY	Не задано	rt
CLEARML_WEB_HOST	CLEARML_WEB_HOST	Не задано	https://app.clear.ml/
CLEARML_API_HOST	CLEARML_API_HOST	Не задано	https://api.clear.ml
CLEARML_FILES_HOST	CLEARML_FILES_HOST	Не задано	https://files.clear.ml
Список расширений	clearml_ext	Не задано	<null>
Список файлов	clearml_files	Не задано	<null>
Игнорировать	ignore	Не задано	true

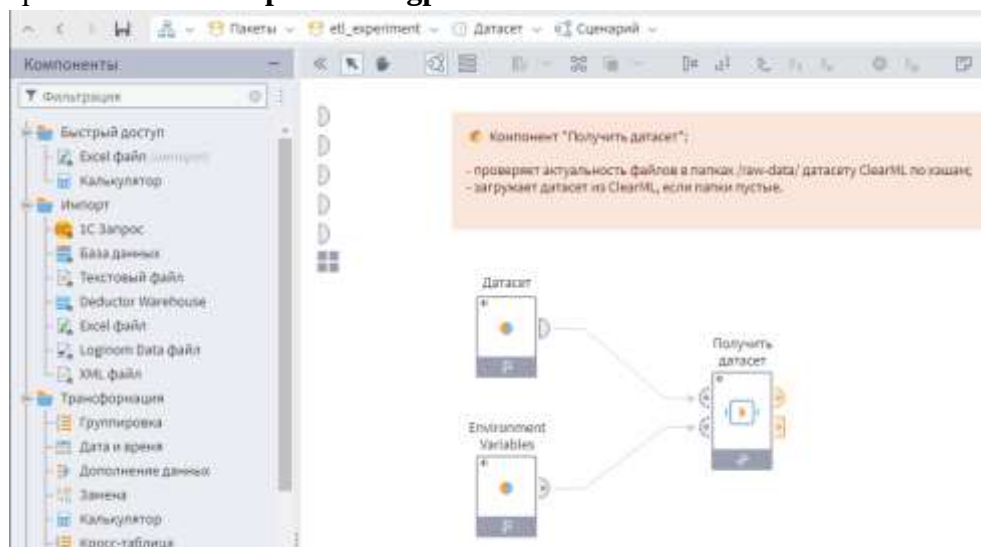
Запустите на выполнение узел **Выгрузка в датасет**. В результате в проект ClearML будут загружены все файлы из папки **raw-data**, а также сценарий **data-prep.lgp**



! Повторный запуск узла загрузит файлы в ClearML с новой версией.

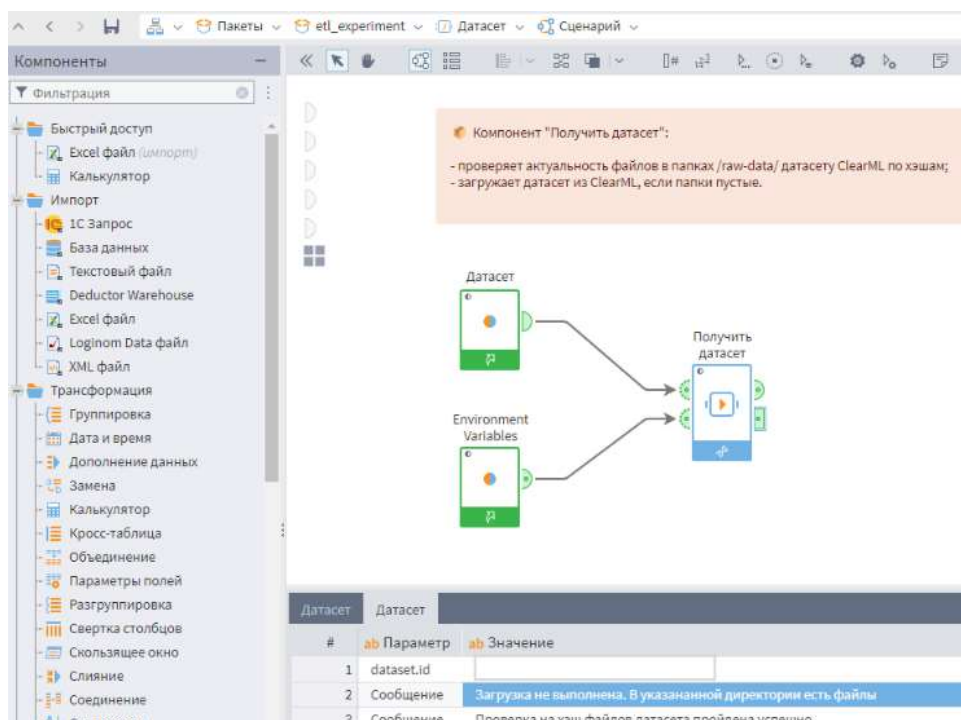
Сценарий ETL-эксперимента

Откройте пакет **etl-experiment.lgp**.

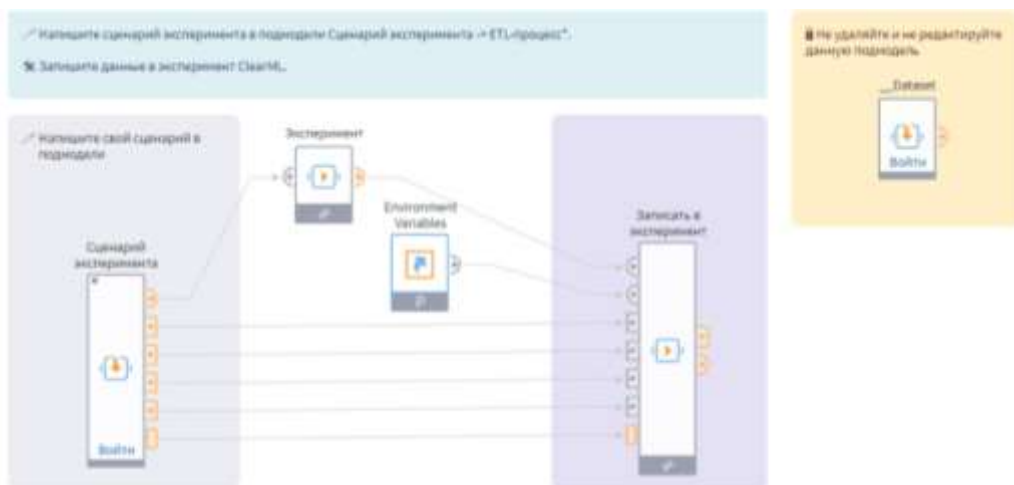


В первом модуле компонент **Получить датасет** позволяет проверить актуальность файлов проекта из папки **raw-data**. Они должны соответствовать файлам, которые хранятся

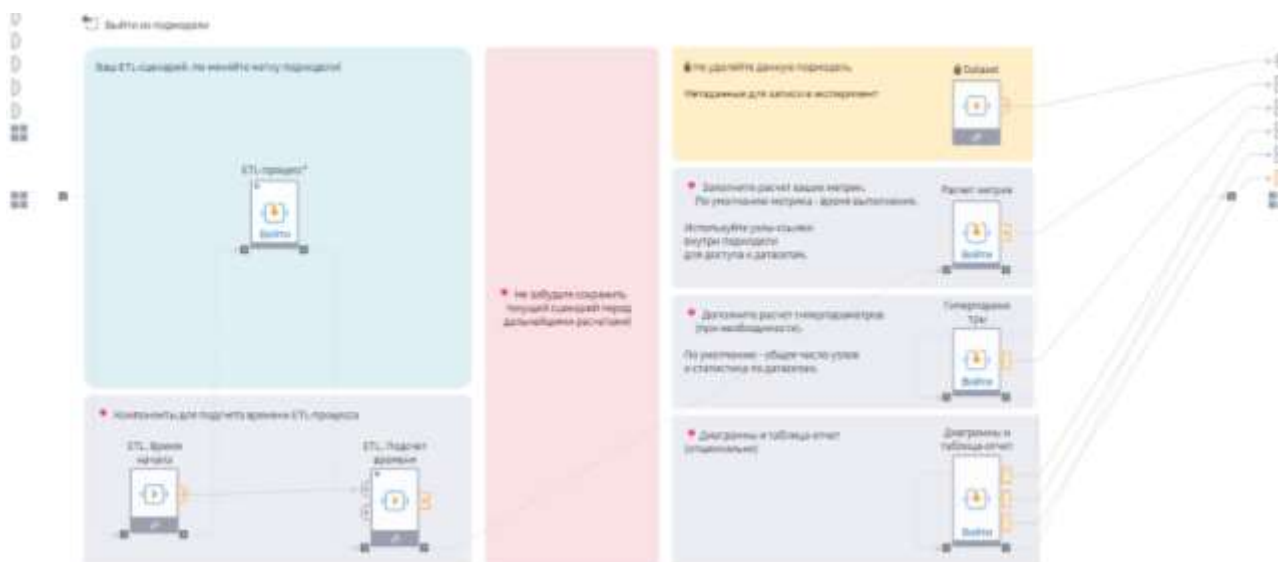
в ClearML. Проверка осуществляется по хэшу. После отработки компонента на его выходном порту будет сформирован результат проверки. Если файлы в папке соответствуют хэшу датасета, они не будут загружены из ClearML.



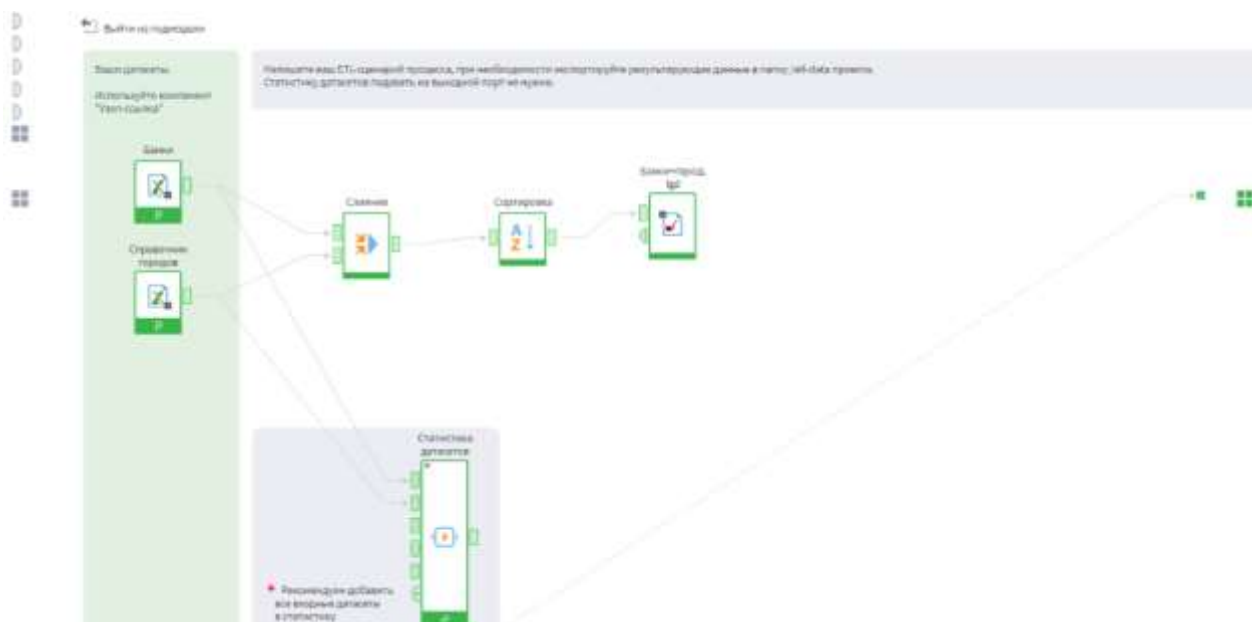
Можно приступить к эксперименту. Откройте одноименный модуль **Эксперимент**.



Войдите в подмодель **Сценарий эксперимента**.



Наконец войдите в подмодель **ETL-процессе**. В ней настройте сценарий обработки датасетов и любых источников, ранее загруженных и настроенных в пакете **data-prep.lgd**.



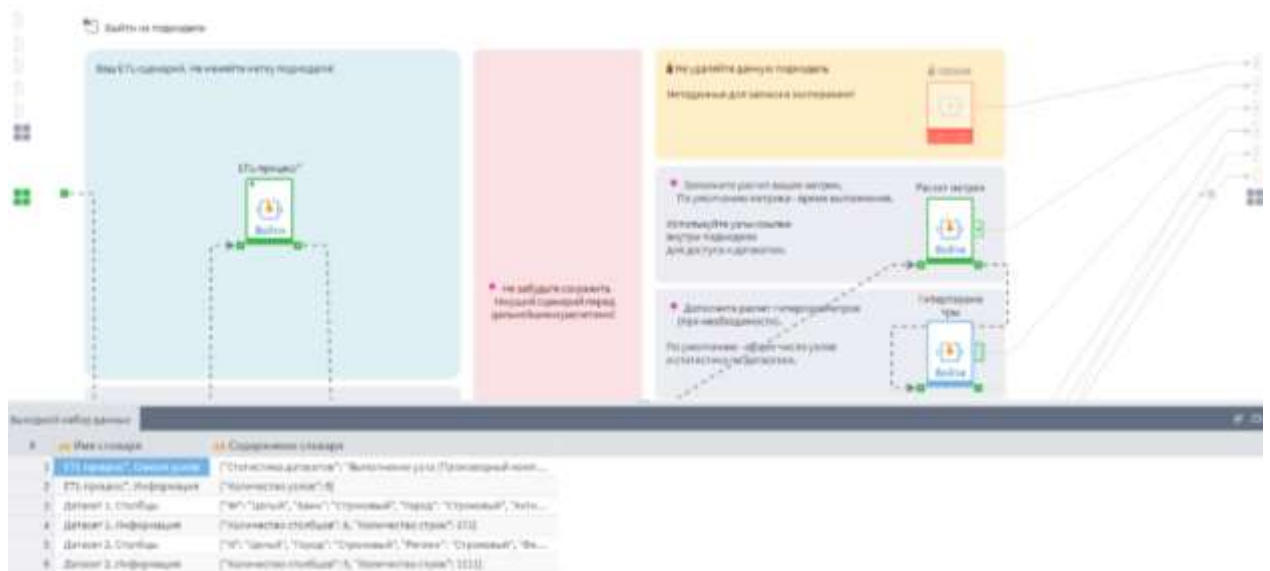
В примере сценария на рисунке идет несложное преобразование двух Excel -файлов в один lgd-файл. Поскольку мы хотим увидеть в карточке эксперимента CleatML структуру «сырых» датасетов, мы отправили их на входы узла **Статистика датасетов**.

Результат нашего мы выгружаем в папку **etl-data** проекта; мы выбрали формат **lgd**.

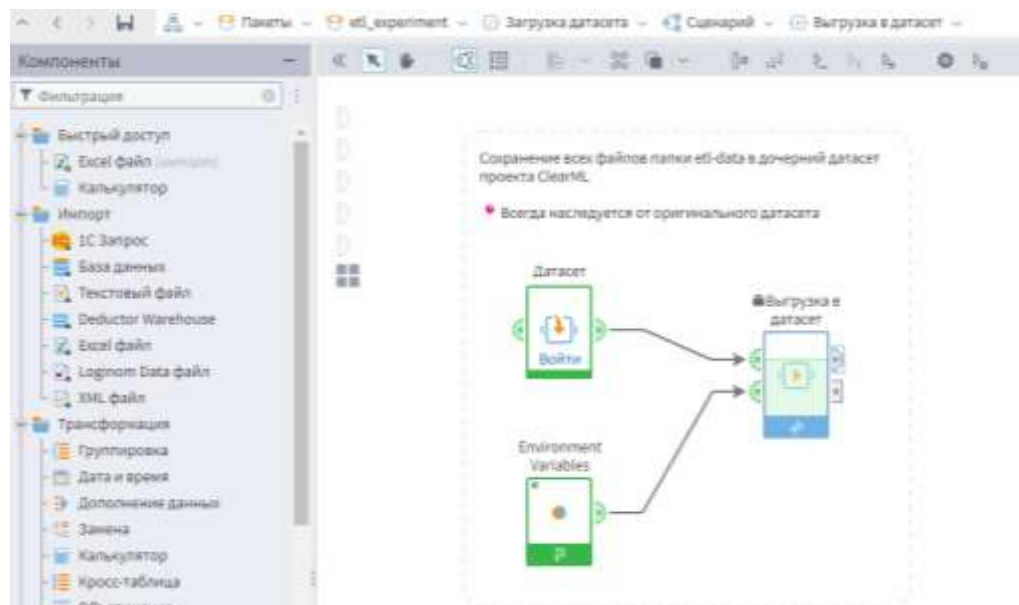
Теперь все готово для записи в эксперимент. По умолчанию в качестве метрики записывается время выполнения (включая выгрузку во внешние источники).

Словари		
#	ab Метрика	9.0 Значение
1	TIME (MS)	394,00
2	TIME (S)	0,00
3	TIME (MIN)	0,00

В состав гиперпараметров включаются количество и наименование узлов процесса, структура и статистика всех датасетов, которые были поданы на вход узла **Статистика датасетов**. В нашем случае это два датасета - **Банки РФ** и **Справочник городов**.



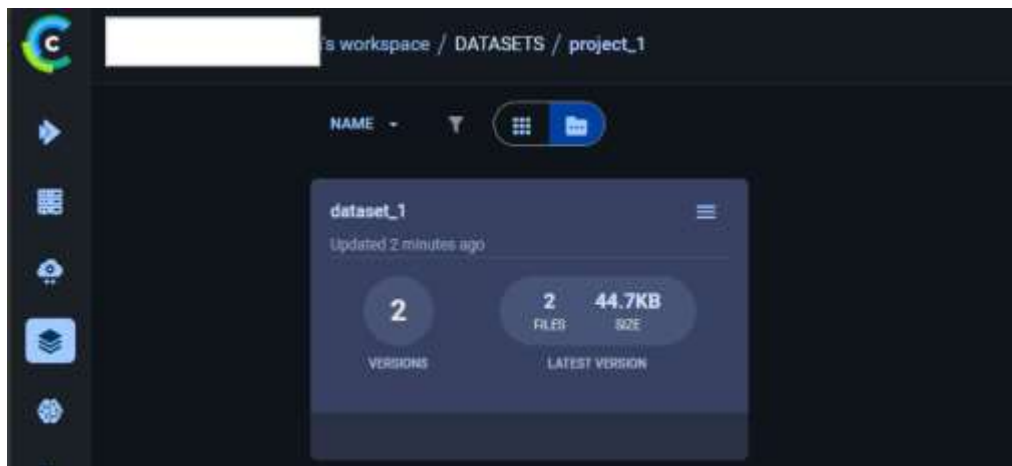
Какой-либо диаграммы или таблицы по умолчанию шаблон ETL-процесс в себя не включает. Если мы хотим сохранить в проект ClearML результирующий датасет, нужно перейти в следующий модуль пакета-шаблона под названием **Загрузка датасета**.



Здесь все настроено для того чтобы сохранить все файлы из папки проекта **etl-data** в дочерний датасет ClearML.

Результат в ClearML

После всех манипуляций мы можем открыть наш проект в ClearML и убедиться в том, что эксперимент со всеми нужными нам артефактами сохранился. В данном случае он назывался **project4**, и мы можем наблюдать его в списке проектов.



Внутри проекта три эксперимента: один - наш и два – автоматически созданных при загрузке датасетов - начального и дочернего.



Теперь перейдем на вкладку Datasets и убедимся, что в с проектом project4 связано две версии датасетов: первая - с тегом **original**, вторая- с тегом **preprocessed**. В последний добавился файл *банки_город.lgd*



Теперь перейдем в карточку эксперимента. В разделе **CONFIGURATION** отражены свойства пакета Loginom и все гиперпараметры, отправленные нами из ETL-процесса.



В разделе **ARTIFACTS** присутствует сам сценарий *etl-experiment.lgp*.



Наконец, на вкладке **SCALARS** отражена метрика - время выполнения сценария ETL-процесса.



Карточка эксперимента. Вкладка SCALARS